

A Bayesian Supplier Payment Agent for Bank-Change Requests

Kushal Kumar K S

Independent course project

<https://github.com/0Kushal0/Supplier-payment-agent>

Abstract

We study a narrow accounts-payable problem: an agent receives an email asking a supplier to change payment bank details and must decide whether to release the payment or hold it. The sender’s true state is hidden, so the agent separates evidence extraction from probabilistic belief updating and action selection. Four hidden states are modeled—genuine, copied, hacked, and spam—with hand-specified priors and clue likelihoods. A local LLM is used only to read three yes/no clues: bank-change request, urgency, and new contact; Bayes’ rule then produces a posterior danger probability. The policy holds a payment when the posterior probability of the copied-or-hacked event exceeds 0.20. On the project’s 100-row labeled test set (70 safe, 30 dangerous), the repository records 96.0% accuracy for the Bayesian agent versus 70.0% for an always-release baseline. The paper also exposes important weaknesses: the priors and likelihoods are not learned from real accounts-payable data, the clue-independence assumption is strong, the LLM can silently fall back to keywords, and the documented vendor-registry evidence is not yet implemented in the running code. The central result is therefore not that the agent is production-ready, but that making hidden uncertainty and error costs explicit gives a useful failure-analysis surface for an AI agent.

1 Introduction

Business email compromise (BEC) and related invoice-redirection attacks can turn ordinary supplier communication into a payment-fraud decision. The FBI describes BEC as a scheme in which attackers impersonate trusted people or organizations and use email to induce fraudulent transfers; the agency’s 2024 BEC advisory reported more than 305,000 domestic and international incidents and more than US\$55 billion in exposed losses across the 2013–2023 period [?].

The specific problem studied here is deliberately smaller: an email asks for a supplier bank-detail change. The agent sees the message, but not the true reason it was sent. A request can be genuine, copied from a real thread, sent from a

compromised mailbox, or be ordinary spam. The decision is asymmetric: releasing a fraudulent payment is potentially far more costly than asking for a legitimate request to be verified.

We therefore design the agent around three explicit objects: (1) hidden states, (2) a belief distribution updated by evidence, and (3) a cost-sensitive action threshold. This is close in spirit to Bayesian decision analysis [?] and to BEC systems that combine multiple behavioral signals rather than relying on a single classifier [?; ?]. Unlike those larger systems, our prototype is intentionally small and inspectable.

The code and test data are available at <https://github.com/0Kushal0/Supplier-payment-agent>. This document is a course preprint; it is not an IJCAI submission and makes no claim of acceptance or submission.

2 Related Work and User Discussions

Prior BEC research includes supervised detectors based on communication history and language features. BEC-Guard separates header/relationship signals from language signals and was evaluated on more than 4,000 attacks [?]. CAPE similarly combines multiple independent detectors over text, images, metadata, and communication context, and highlights practical constraints such as data scarcity, explainability, and continuous adaptation [?]. A recent systematic review categorizes BEC defenses into rule-based, machine-learning, and NLP approaches and identifies data availability and evolving attacker behavior as open challenges [?]. Our prototype sits at a different point in this design space: it does not train a fraud classifier; it turns a small set of observed clues into an explicit posterior over named hidden states.

Public practitioner discussions reinforced the importance of verification outside the email channel. In r/sysadmin incident reports, compromised vendor conversations have been used to request bank-detail changes, and authors have described missing callback confirmation as a critical error. These reports influenced the prototype’s emphasis on bank-change requests as a high-value clue and on retaining a human hold path rather than auto-rejecting messages.

The discussions also exposed a deployment tension for LLM-based evidence extraction. Community members discussing local LLM use for sensitive enterprise data repeatedly raised privacy and operational-control concerns. The project therefore uses a local Ollama model and restricts the LLM to

clue extraction; the probabilistic policy remains deterministic and inspectable.

3 Agent Design

3.1 Agent Input

The input is one supplier email requesting, or appearing to request, a payment-account action. The current test set contains 100 labeled examples. The implementation reads only the email text and produces a receipt containing the observed clues, the posterior distribution, and the selected action.

3.2 Hidden States

The agent models four mutually exclusive hidden states:

State	Interpretation
genuine	legitimate supplier request
copied	copied/impersonated communication
hacked	compromised supplier mailbox
spam	unsolicited or otherwise malicious message

These states are not directly observed. The distinction matters because the same email text can arise from different underlying causes.

3.3 Evidence and Action

The current implementation asks an LLM three binary questions: whether the message asks for changed bank details (`bank_change`), whether it creates urgency (`urgency`), and whether it introduces an unfamiliar contact channel (`new_contact`). If the local model is unavailable or returns invalid JSON, the code falls back to hand-written keyword matching.

The agent defines the event *danger* as the union of the copied and hacked states. It can take two actions: RELEASE or HOLD. The prototype deliberately keeps a human-controlled hold path instead of claiming that the model can safely auto-reject a supplier.

4 Probability Model and Decision Rule

The prior in the current code is

$$P(s) = \{0.80, 0.08, 0.04, 0.08\}$$

for genuine, copied, hacked, and spam respectively. These values are explicitly marked in the code as hand-typed assumptions rather than measured base rates.

For evidence vector $e = (e_1, e_2, e_3)$, the implementation assumes the clues are conditionally independent given the hidden state. The posterior is therefore computed as

$$P(s | e) = \frac{P(s) \prod_i P(e_i | s)}{\sum_{s'} P(s') \prod_i P(e_i | s')}. \quad (1)$$

This is the usual Bayesian update form [?]. The independence assumption is a limitation, not an established fact about email evidence.

The danger probability is

$$D(e) = P(\text{copied} | e) + P(\text{hacked} | e). \quad (2)$$

The current policy is

$$a(e) = \begin{cases} \text{HOLD} & \text{if } D(e) > 0.20, \\ \text{RELEASE} & \text{if } D(e) \leq 0.20. \end{cases} \quad (3)$$

Decision theory provides a useful interpretation. Let C_{FR} be the cost of a false RELEASE on a dangerous request and C_{FH} the cost of a false HOLD on a genuine request. With only these two error costs, a Bayes-risk threshold would be

$$p^* = \frac{C_{FH}}{C_{FR} + C_{FH}}. \quad (4)$$

Thus a lower threshold is justified when false RELEASE is much more expensive. In this project the repository records only a qualitative cost asymmetry, so the 0.20 threshold is a policy choice rather than a threshold derived from measured monetary costs.

For a worked case, the email “Urgent: we changed our bank account today...” produces the evidence vector (1, 1, 0). Applying the update gives posterior beliefs of approximately 0.238 genuine, 0.190 copied, 0.421 hacked, and 0.150 spam, so $D = 0.611$ and the policy selects HOLD.

The decision record also explores a second evidence source, a vendor-registry check. Starting from the previous posterior and multiplying by the recorded likelihoods for a “not-found” registry result shifts mass toward spam and copied states. This demonstrates that new evidence can change the explanation even when the action stays the same. Importantly, this registry evidence is documented in the decision record but is not yet part of the running `posterior()` function.

5 Test Method

The repository evaluates the prototype on a 100-row labeled email set containing 70 safe and 30 dangerous examples. The baseline, P0, always returns RELEASE. The V1 agent reads the clues, computes the posterior, and applies the threshold rule. Precision, recall, and F1 are measured for the dangerous class.

The reported V1 number in the repository comes from a local-LLM run. A 96.0% accuracy over 100 cases with 70 safe cases, 100% dangerous precision, and 86.7% dangerous recall implies 70 true negatives, 26 true positives, 4 false negatives, and 0 false positives. Those counts exactly reproduce the reported 96.0% accuracy and 92.9% F1.

A deterministic keyword replay is also informative. Several dangerous examples that contain only the bank-change clue yield $D \approx 0.140$, just below the 0.20 HOLD threshold. The resulting missed cases explain the documented recall failure mode.

6 Results

The baseline’s 70.0% accuracy is exactly its safe-class proportion because it releases every message. V1 adds 26 correct dangerous holds without introducing a false hold in the recorded run, raising accuracy by 26 percentage points. The main trade-off is four dangerous false releases, concentrated in weak single-clue cases.

Policy	Accuracy	Precision	Recall	F1
P0 always RELEASE	70.0%	0.0%	0.0%	0.0%
V1 posterior threshold	96.0%	100.0%	86.7%	92.9%

Table 1: Repository-reported results on the 100-example test set. The V1 metrics are internally consistent with 70 true negatives, 26 true positives, 4 false negatives, and 0 false positives.

Failure	What happens
Single weak clue	Bank-change alone gives $D \approx 0.140 < 0.20$, so some dangerous requests are released.
Silent LLM fallback	When Ollama fails or returns bad JSON, the code silently uses keyword matching while the run can still be labeled as an LLM run.
Evidence/code mismatch	The decision record includes vendor-registry evidence, but the implemented posterior currently uses only the three email clues.
Correlated clues	Bank-change and urgency can be strongly related, yet the update multiplies their likelihoods as if conditionally independent.
Prior/likelihood uncertainty	The numerical prior and clue likelihoods are assumptions, so posterior values may be overconfident even when the arithmetic is correct.

Table 2: Observed and documented failure modes in the current prototype.

The result should not be read as evidence of production-level detection quality. The test set is small, the priors and likelihoods are hand-specified, and the LLM evidence extractor itself is not evaluated for calibration or observation error. The most defensible conclusion is narrower: an explicit probabilistic action layer can outperform a trivial baseline on the project’s constructed test set while making concrete failure modes visible.

7 Failure Analysis

These are not cosmetic issues. The silent fallback can change the evidence model without telling the operator. The vendor-registry mismatch can make a hand-worked example appear stronger than the actual software. The independence assumption can double-count related signals. Finally, the small test set can make 96% accuracy look much more certain than it is.

8 Limitations, Ethics, and Human Control

The system is intentionally a prototype. It uses no real vendor master data, no live bank verification service, and no measured fraud base rates. The test emails are small and repetitive. The current code also ignores uncertainty in the LLM’s own clue-reading step: the posterior treats each reported clue as observed with certainty.

Ethically, the main risk is automation bias. A high posterior should not be treated as proof that an email is malicious, and a low posterior should not be treated as proof that it is safe. The model should therefore support a human AP or finance workflow rather than silently changing payment details. A practical deployment would require independent verification for bank-detail changes, audit logs, access controls, monitored fallback behavior, and clear escalation rules.

Human control is built into the action space through HOLD, but the current prototype still has an overly sharp binary decision. Future versions should consider a three-way policy such as RELEASE / MANUAL VERIFY / STOP, where the middle state can request additional evidence such as a callback to a known supplier number or a vendor-master comparison.

AI-use statement. Generative AI tools were used during software review, debugging, drafting, and editing. The author remained responsible for the problem definition and final artifact. Citations were checked against source pages, equations were recomputed, the reported metric table was checked for internal consistency, and the final PDF was visually inspected after compilation.

Contribution statement. This is a sole-author course project. Kushal Kumar K S contributed to problem selection, public discussions, agent design, software development, tests, citation checks, all preprint sections, and PDF checks.

9 Conclusion and New Questions

This project shows a useful design pattern for an AI agent operating under uncertainty: name the hidden states, write down a prior, extract a small amount of observable evidence, update beliefs explicitly, and map the resulting risk to an action through an inspectable policy. On the project’s 100-case test set, that design substantially outperforms an always-release baseline, but the weaknesses are equally important: the numbers are assumed, the clues are not independent, the LLM can fail silently, and one of the strongest evidence sources in the hand-worked analysis is not implemented.

The next questions are therefore more interesting than simply asking for a higher accuracy number. How should priors and likelihoods be estimated from real AP data? How much does explicit modeling of LLM observation error change the posterior? Can a value-of-information policy decide when a callback or vendor-registry query is worth the extra friction? And can a three-way human-in-the-loop policy reduce false releases without overwhelming finance teams with manual reviews?

Project link: <https://github.com/0Kushal0/Supplier-payment-agent>

References